Greenhouse Management
Validation + Verification

Jane Marie Pareja

April 10, 2026

Contents

1 Validation **2**
1.1 Testing Objectives 2
1.2 Test Environment 2
1.3 Test Cases 3
1.3.1 Data Retrieval Test 3
1.3.2 Dashboard Display Test 3
1.3.3 Chart Functionality Test 5
1.3.4 Threshold Alert Test 7
1.3.5 Mobile Responsiveness Test 7
1.3.6 Performance Test 9
1.3.7 Validation Results 9
1.4 Unit Testing 9

2 Verification **11**

3 Future Scope **12**

4 Conclusion **13**

1 Validation

Validation is the assurance that a product, service, or system meets the needs of the customer and other identified stakeholders.

In this document, I intend to work on revising the web-based greenhouse dashboard to reflect the feedback received, i.e., applying updates and improvements towards a better result or IT product.

This project aims to build a web-based platform to help farmers monitor their crops. By centralizing critical environmental data, i.e., temperature, humidity, soil moisture, and light, into an accessible dashboard, the system enables users to make data-driven decisions that result in harvest optimization while reducing production costs.

1.1 Testing Objectives

The main objectives of testing are:

- To verify that all system functions work as expected
- To ensure accurate data collection and storage
- To validate data processing and visualization
- To confirm that the user interface is responsive and user-friendly
- To identify and fix errors or bugs

1.2 Test Environment

Component	**Description**
Backend	Flask (Python)
Database	SQLite
Frontend	HTML, CSS, JavaScript
Browser	Chrome, Edge

Table 1: Test Environment

1.3 Test Cases

1.3.1 Data Retrieval Test

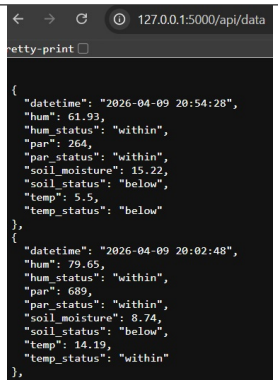
Test Case	Description
Objective	Check if data is correctly retrieved from the database
Input	Request data from /api/data
Expected Result	Data is returned in JSON format
Actual Result	
Status	Pass

Table 2: Data Retrieval Test

1.3.2 Dashboard Display Test

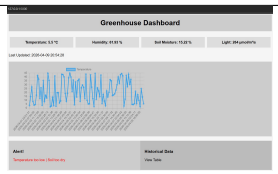
Test Case	Description
Objective	Verify that data is displayed on the dashboard
Input	Load the homepage
Expected Result	Tiles show latest values correctly
Actual Result	
Status	Pass

Table 3: Data Retrieval Test

Figure 1 shows the actual result/product as presented in my realization document. I asked Raja for feedback and she looked at it. She said it was good, however, if I could make the graph bigger to cover the entire container reserved for it, then it would be even better.

I agreed with her suggestion. I then worked on it, changed the dimensions of the graph in my template and it looked better as Figure 2 illustrates.

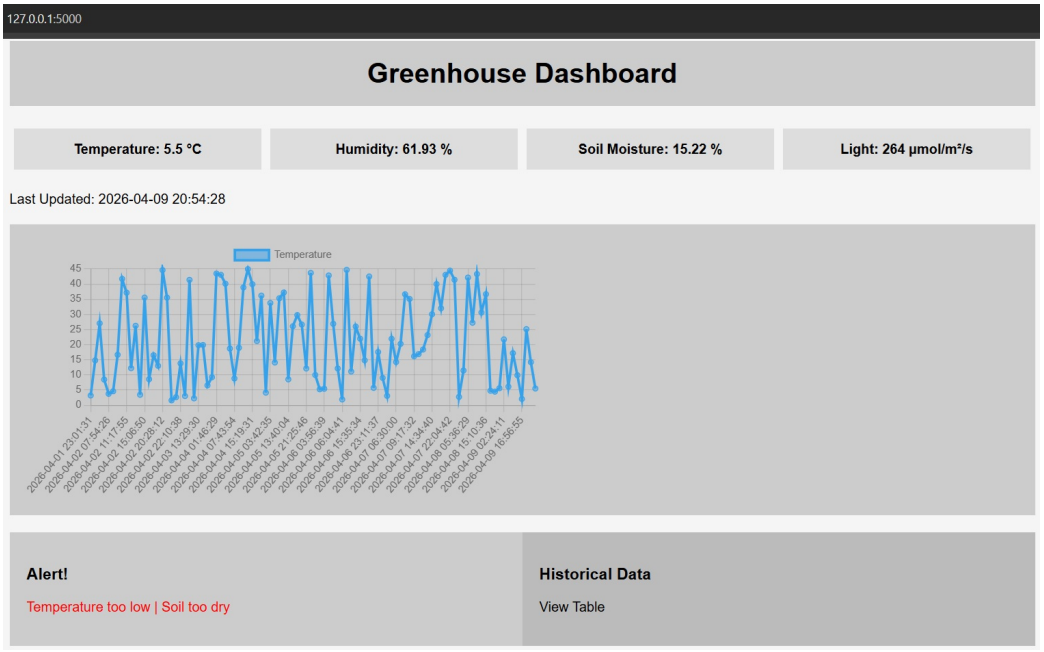


Figure 1: Greenhouse Dashboard Main Page

She also asked for the X and Y labels on the chart to be made more visible and readable, and I thought that was actually a great idea. I have been meaning to put more colors on my dashboard to make it look more lively. I told myself that I will add more colors to it after making sure that the requirements are working, which I did. Now it would be wonderful if I could make it more attractive than how it looked in Figure 2.

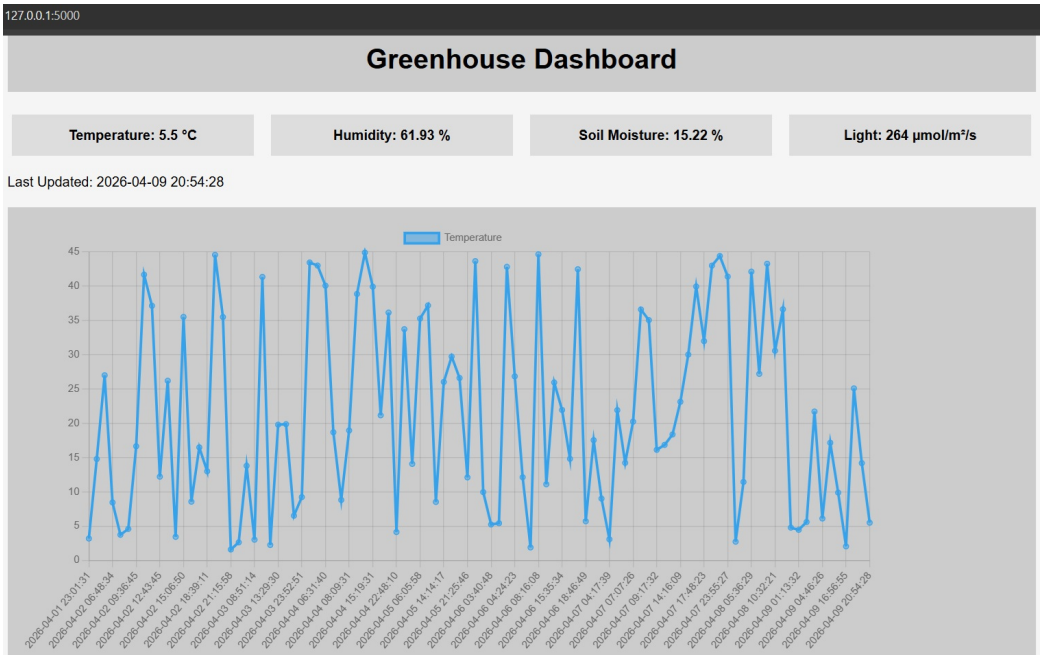


Figure 2: The Chart made Wider

1.3.3 Chart Functionality Test

Test Case	Description
Objective	Ensure charts display correct data
Input	Click on different cards
Expected Result	Chart updates based on selected parameter
Actual Result	See Figure 4
Status	Pass

Table 4: Chart Functionality Test

I did not just work on making the font more visible, I also worked on making the chart a bit more colorful, namely different line colors per chart as demonstrated in Figure 3. This looks great, however, this is not how I wanted it to be. Because here, the cards stopped working. When you clicked on them, they are not responding. If you would like to see the charts, then you will have to click on the legends, that is, those small colorful rectangles.

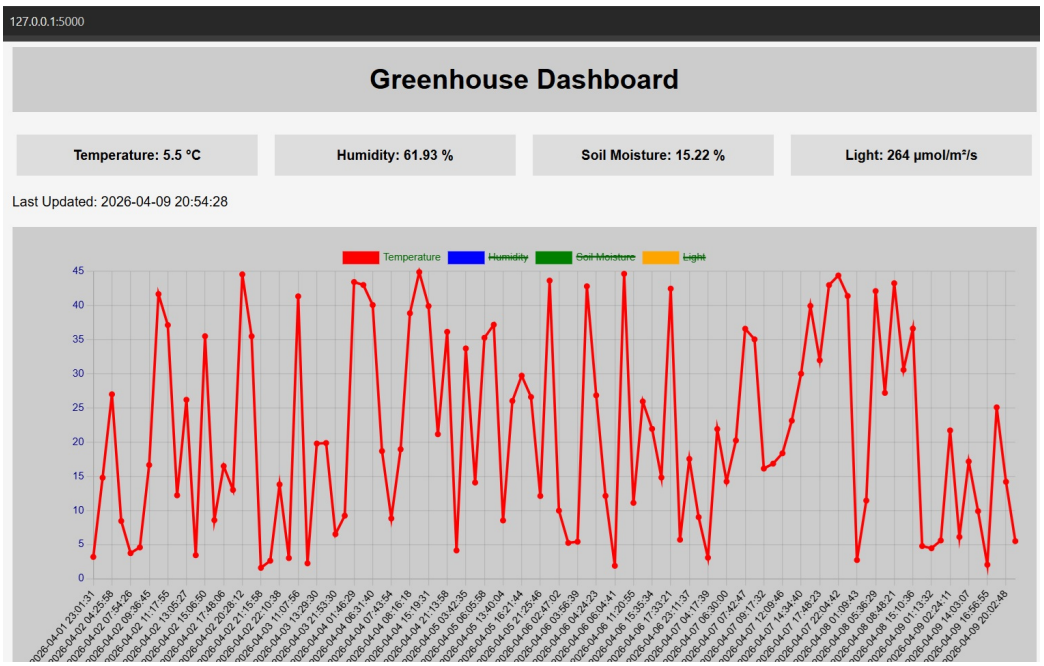
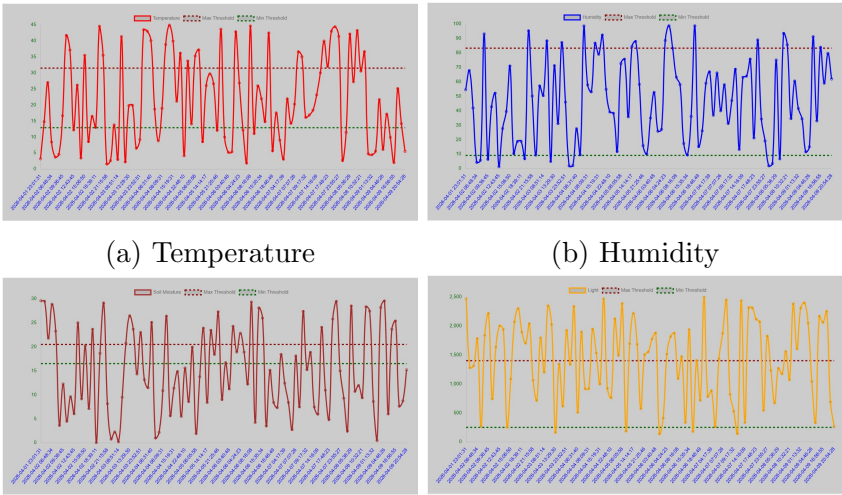


Figure 3: One Chart - Multiple Legends

What I did, I worked on making sure that the cards works again and also added the threshold indicator lines to show which numbers fall within the normal threshold as shown in Figure 4.



(a) Temperature (b) Humidity

(c) Soil Moisture (d) Light (PAR)

Figure 4: Different Charts with Threshold Indicators

1.3.4 Threshold Alert Test

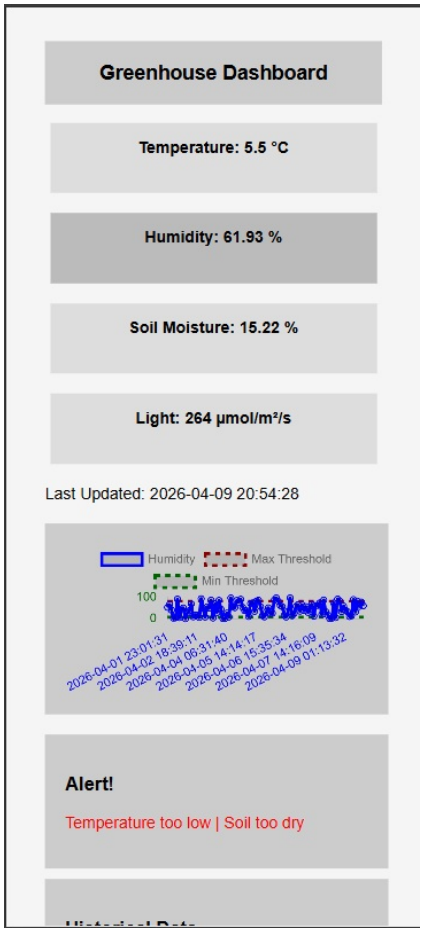
Test Case	**Description**
Objective	Check if threshold lines appear correctly
Input	Load chart with thresholds
Expected Result	Max and Min lines are displayed
Actual Result	See Figure 4
Status	Pass

Table 5: Threshold Alert Test

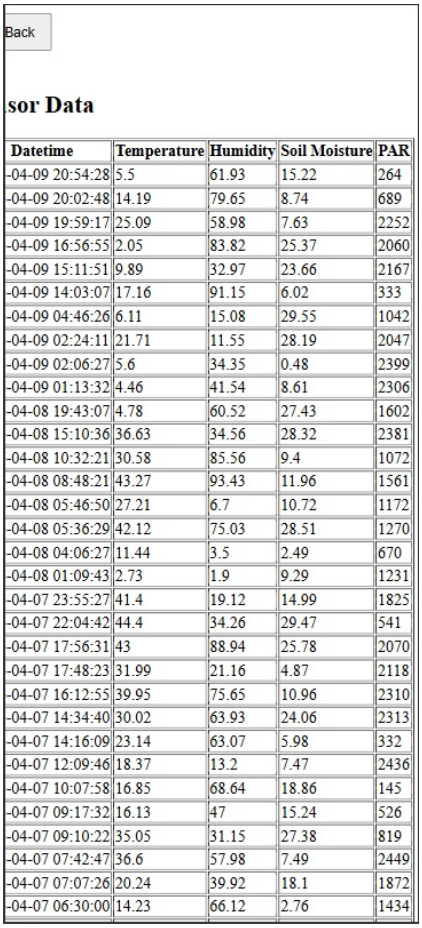
1.3.5 Mobile Responsiveness Test

Test Case	**Description**
Objective	Verify mobile view functionality
Input	Open site on mobile or resize browser
Expected Result	Layout adjusts properly
Actual Result	See Figure 5
Status	Pass

Table 6: Threshold Alert Test



(a) Main Page



Back

sor Data

Datetime	Temperature	Humidity	Soil Moisture	PAR
-04-09 20:54:28	5.5	61.93	15.22	264
-04-09 20:02:48	14.19	79.65	8.74	689
-04-09 19:59:17	25.09	58.98	7.63	2252
-04-09 16:56:55	2.05	83.82	25.37	2060
-04-09 15:11:51	9.89	32.97	23.66	2167
-04-09 14:03:07	17.16	91.15	6.02	333
-04-09 04:46:26	6.11	15.08	29.55	1042
-04-09 02:24:11	21.71	11.55	28.19	2047
-04-09 02:06:27	5.6	34.35	0.48	2399
-04-09 01:13:32	4.46	41.54	8.61	2306
-04-08 19:43:07	4.78	60.52	27.43	1602
-04-08 15:10:36	36.63	34.56	28.32	2381
-04-08 10:32:21	30.58	85.56	9.4	1072
-04-08 08:48:21	43.27	93.43	11.96	1561
-04-08 05:46:50	27.21	6.7	10.72	1172
-04-08 05:36:29	42.12	75.03	28.51	1270
-04-08 04:06:27	11.44	3.5	2.49	670
-04-08 01:09:43	2.73	1.9	9.29	1231
-04-07 23:55:27	41.4	19.12	14.99	1825
-04-07 22:04:42	44.4	34.26	29.47	541
-04-07 17:56:31	43	88.94	25.78	2070
-04-07 17:48:23	31.99	21.16	4.87	2118
-04-07 16:12:55	39.95	75.65	10.96	2310
-04-07 14:34:40	30.02	63.93	24.06	2313
-04-07 14:16:09	23.14	63.07	5.98	332
-04-07 12:09:46	18.37	13.2	7.47	2436
-04-07 10:07:58	16.85	68.64	18.86	145
-04-07 09:17:32	16.13	47	15.24	526
-04-07 09:10:22	35.05	31.15	27.38	819
-04-07 07:42:47	36.6	57.98	7.49	2449
-04-07 07:07:26	20.24	39.92	18.1	1872
-04-07 06:30:00	14.23	66.12	2.76	1434

(b) Table Page

Figure 5: Mobile View Dashboard

After adding colors to the chart, and upon reviewing the requirements, I had enumerated in the design of this project. I realized that I was missing the mobile view feature of this dashboard where users can also view the dashboard on their phones. With the help of AI [1], I was able to learn how it can be done and applied it to my project as Figure 5 conveys.

This mobile view works exactly like the web view where users can click on the different cards like Temperature, Humidity, Soil Moisture, and Light. However, the graph looks so small and if given I have more time left, I might be able to make it look nicer.

[1] Greenhouse IT Integration

1.3.6 Performance Test

Test Case	Description
Objective	Check system speed and responsiveness
Input	Load dashboard with data
Expected Result	Page loads quickly without lag
Actual Result	See Performance Test Video
Status	Pass

Table 7: Performance Test

1.3.7 Validation Results

The system was tested under different conditions. Most functionalities worked as expected, including:

- Data retrieval from the database
- Dashboard Updates
- Chart Visualization and interaction
- Alerts and notifications display
- Threshold display

The greenhouse management system has been successfully tested and validated. The system meets its main objectives by providing accurate data, useful visualizations, and a user-friendly interface. It is suitable for small-scale greenhouse monitoring and demonstrates the effective use of IT solutions in agriculture.

1.4 Unit Testing

Unit testing is the process of testing individual parts (units) of the system to ensure they work correctly. In this project, unit testing was performed using pytest. Pytest is used to automatically test different parts of the system to ensure they work correctly.

Example Unit Tests (test_app.py)

- Test 1: Flask App is running and home page loads
- Test 2: API Returns data
- Test 3: Data not Empty
- Test 4: Check for required fields
- Test 5: Database Connection

I run the test in terminal and received the expected output as seen in Figure 6.

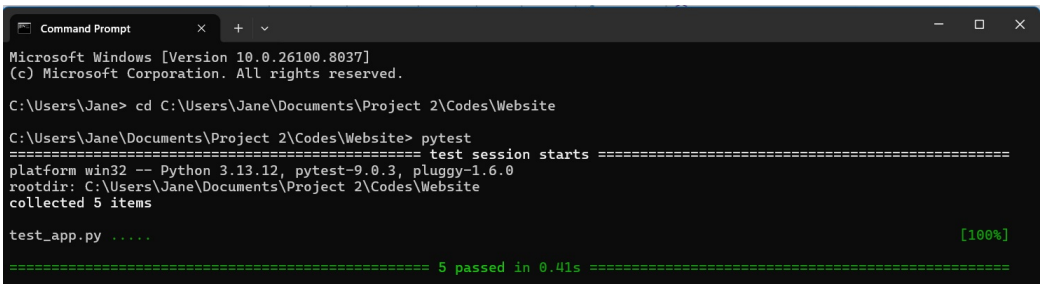


Figure 6: Unit Test - Pytest

Unit testing was successfully implemented using pytest. The tests verified that the backend, API, and database function correctly. This ensures that the greenhouse management system is reliable and free from major errors.

2 Verification

Verification is the evaluation of whether or not a product, service, or system complies with a regulation, requirement, specification, or imposed condition.

In the analysis and advice document of the project, I listed certain functional and non-functional requirements that the product of this project should satisfy upon its completion. I enumerate these requirements as a checklist to show whether the final product met those requirements or not, as follows.

- ☑ Data Collection - the system shall collect data such as humidity, light, temperature, and soil moisture, and shall support real-time or simulated sensor data input.

  I used a mock data on this project to simulate sensor data for temperature, humidity, soil moisture, and light.

- ☑ Data Storage - the system shall store collected data and save the data with timestamps for tracking overtime.

  The mock data also include data and time for the timestamps and it was stored in a database via SQL.

- ☑ Data Processing - the system shall clean and preprocess data and shall organize data for analysis.

  This has been done via Python using SQLite3.

- ☑ Data Analysis - the system shall perform time-series analysis to show trends, shall detect anomalies, and shall generate alerts when thresholds are exceeded.

  This has also been done together with processing the data via SQLite3 in Python.

- ☑ User Interface (Dashboard) - the system shall display real-time data on a dashboard. shall show graphs, and shall allow users to view historical data.

  Python has used with Flask, HTML, CSS and Javascript to build the dashboard. They have the cards for the sensor data, has the chart, and users can also view the historical data in a table format.

- ☑ Alerts and Notification - the system shall notify users when conditions exceeds safe limits and alerts shall be visible on the dashboard.

The dashboard has an alert box that warns the user whenever the data reached outside the threshold.

- ☐ Performance - the system should update data in real-time and should respond quickly to user actions.

  This non-functional was not met because the I used a mock data to simulate the sensor data for temperature, humidity, soil moisture, light. Thus, they were not real-time but they do respond quickly to user actions.

- ☑ Usability - the system should have a simple and user-friendly interface, and the dashboard should be easy to understand for non-technical users.

- ☑ Reliability - the system should operate consistently without frequent failures, and should handle errors without crashing.

- ☑ Maintainability - the system should be easy to maintain and modify. Code should be easy to organized and documented.

  I have added markdown notes in the template to label what a certain group of codes does hopefully to help the user understand how they work.

- ☑ Compatibility - the system should work on common web browsers, and the dashboard should be accessible on both desktop and mobile devices.

3 Future Scope

Looking ahead, the Greenhouse Management project will grow from a simple dashboard into a fully smart and independent system. I asked some peers to use the dashboard and sent them a peer feedback form via google form [2]. They give some really useful suggestions that can be applied as future scopes of this project.

One of which involves using machine learning models that can look at data to predict exactly what a plant needs before a problem even starts. It includes real time alerts that should notify the users when something falls outside the threshold. Instead of just showing information, the greenhouse will use advanced automation systems to take action on its own, like automatically

[2] Peer Feedback Summary Result

turning on water or adjusting the lights.

To help even larger farms, the project can expand into large-scale sensor network installations, allowing farmers to monitor thousands of plants at once with high precision. The ultimate vision is a full-scale commercial deployment, bringing these powerful tools to the global market to make farming easier, cheaper, and more productive for everyone.

4 Conclusion

This greenhouse management project shows how technology can make farming easier and more efficient. By collecting data such as temperature, humidity, soil moisture, and light, the system helps farmers understand what is happening inside the greenhouse at any time.

The use of a database, data analysis, and a web dashboard allows data monitoring and quick decision-making. Features like charts and alerts helps detect problems early, which can protect crops and improve productivity.

Overall, this project demonstrates that using simple IT solutions can improve sustainability, reduce manual work, and increase the success of greenhouse farming.